Quick Start Guide - YouTube Shorts Recommendation System

Get Started in 5 Minutes!

Option 1: One-Command Start (Recommended)

```
# Clone and setup everything
git clone <your-repo>
cd youtube-shorts-recommendation
make full-pipeline
```

Option 2: Step-by-Step Setup

1. Install Dependencies

```
# Create virtual environment
python3.11 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# Install packages
pip install -r requirements.txt
```

2. Generate Data & Train Model

```
# Generate synthetic data
python src/data_generator.py

# Validate data quality
```

```
python src/data_validation.py

# Train the ML model
python src/train.py
```

3. Start the API

```
# Start API server
uvicorn src.api:app --host 0.0.0.0 --port 8000 --reload
```

4. Launch UI (Optional)

```
# In a new terminal
streamlit run src/streamlit_app.py --server.port 8501
```

PLAY AROUND HERE:

Streamlit UI (Interactive Interface)

→ **http://localhost:8501** ←

Features:

- **Single Prediction**: Test individual user-video predictions
- **Batch Prediction**: Upload CSV files for bulk processing
- **Analytics Dashboard**: View system metrics
- 🤖 **Model Info**: Explore model details and feature importance

API Endpoints (For Developers)

Health Check

```
curl http://localhost:8000/health
```

Single Prediction

```
curl -X POST "http://localhost:8000/predict" \
 -H "Content-Type: application/json" \
 -d '{
 "user_id": "user_123",
 "video_id": "video_456",
 "watch_time": 45.5,
 "hour_of_day": 14
 }'
```

Batch Prediction

```
curl -X POST "http://localhost:8000/predict/batch" \
 -H "Content-Type: application/json" \
 -d '{
 "requests": [
 {"user_id": "user_1", "video_id": "video_100", "watch_time": 30.0},
 {"user_id": "user_2", "video_id": "video_200", "watch_time": 60.0}
 ]
 }'
```

Model Information

```
curl http://localhost:8000/model/info
curl http://localhost:8000/metrics/features
```

Docker Deployment (Production Ready)

Quick Docker Start

```
# Build and run everything
make deploy

# Or manually:
docker-compose up -d
```

Access Points:

- **API**: http://localhost:8000
- **Streamlit UI**: http://localhost:8501
- **MLflow**: http://localhost:5000
- **Prometheus**: http://localhost:9090
- **Grafana**: http://localhost:3000 (admin/admin123)

Production Deployment

```
# With monitoring stack
docker-compose up -d

# Check status
docker-compose ps
docker-compose logs -f api
```

🧪 Testing

API Tests

```
# Run comprehensive API tests
python tests/test_api.py

# Or use pytest
pytest tests/test_api.py -v
```

Load Testing

```
# Install locust
pip install locust

# Run load test
locust -f locustfile.py --host http://localhost:8000 \
 --headless -u 10 -r 2 -t 60s
```

Sample Data for Testing

CSV Format for Batch Prediction

```
user_id,video_id,watch_time,hour_of_day
user_1,video_100,45.5,14
user_2,video_200,30.0,9
user_3,video_300,120.0,22
```

Expected Response Format


```
{
 "user_id": "user_123",
 "video_id": "video_456",
 "probability": 0.742,
 "confidence": "high",
 "model_version": "1.0.0",
 "response_time_ms": 45.2,
 "timestamp": "2024-01-15T14:30:45"
}
```


Useful Commands

Development

```
# Full development setup
make dev

# Train new model
make train

# Run all tests
make test

# Format code
make format

# View logs
make logs
```

Monitoring

```
# Check service health
make health

# View metrics
make monitor

# Create backup
make backup
```

Troubleshooting

Common Issues

1. Port Already in Use

```
# Kill process on port 8000
lsof -ti:8000 | xargs kill -9

# Or use different port
uvicorn src.api:app --port 8001
```

2. Module Not Found

```
# Ensure virtual environment is activated
source venv/bin/activate
pip install -r requirements.txt
```

3. No Trained Model

```
# Train the model first
python src/train.py
```

4. Redis Connection Error

```
# Start Redis (if using Docker)
docker run -d -p 6379:6379 redis:7-alpine

# Or disable Redis in api.py (set redis_client = None)
```

Performance Tips

1. Faster Training

```
# Use optimized training
python src/train.py --optimize=false # Skip hyperparameter optimization
```

2. Production Settings

```
# Use multiple workers
uvicorn src.api:app --workers 4 --host 0.0.0.0 --port 8000
```

3. Memory Optimization

```
# For limited memory
export PYTHONHASHSEED=0
ulimit -m 2000000 # Limit to 2GB
```

What You'll See

1. Training Output

```
Generating synthetic data for YouTube Shorts recommendation system...
Generated 10000 users
Generated 10000 videos
Generated 100000 interactions

Starting training pipeline...
Training baseline models...
logistic_regression - Accuracy: 0.7234, AUC: 0.7891
random_forest - Accuracy: 0.7456, AUC: 0.8123
lightgbm - Accuracy: 0.7689, AUC: 0.8334

Best model: lightgbm_optimized with AUC: 0.8456
 Training complete!
```

2. API Responses


```
{
 "user_id": "user_123",
 "video_id": "video_456",
 "probability": 0.742,
 "confidence": "high",
 "response_time_ms": 45.2
}
```


3. Streamlit Interface

- Interactive sliders for watch time
- Real-time predictions with probability gauges

- Batch upload with CSV processing
- Analytics charts and metrics
- Model explainability features

Next Steps

Experimentation Ideas

1. **Try Different Users**: Test with various user_ids (user_1 to user_9999)
2. **Vary Watch Times**: See how predictions change with different engagement levels
3. **Time Analysis**: Test different hours_of_day for temporal patterns
4. **Batch Processing**: Upload CSV files with multiple predictions
5. **Load Testing**: Run Locust tests to see system performance

Customization

1. **Model Tuning**: Modify hyperparameters in `src/train.py`
2. **Feature Engineering**: Add new features in `src/feature_engineering.py`
3. **API Extensions**: Add new endpoints in `src/api.py`
4. **UI Enhancements**: Customize the Streamlit interface

Production Deployment

1. **Cloud Deploy**: Use Kubernetes manifests for cloud deployment
2. **Monitoring**: Set up Grafana dashboards for production monitoring
3. **CI/CD**: Enable GitHub Actions for automated deployment
4. **Scaling**: Configure horizontal pod autoscaling

Support

- **Logs**: Check `docker-compose logs -f api` for debugging
- **Health**: Visit http://localhost:8000/health for system status
- **Metrics**: Use http://localhost:9090 for Prometheus metrics
- **Documentation**: API docs at http://localhost:8000/docs

** Ready to test? Start with the Streamlit UI at http://localhost:8501!**